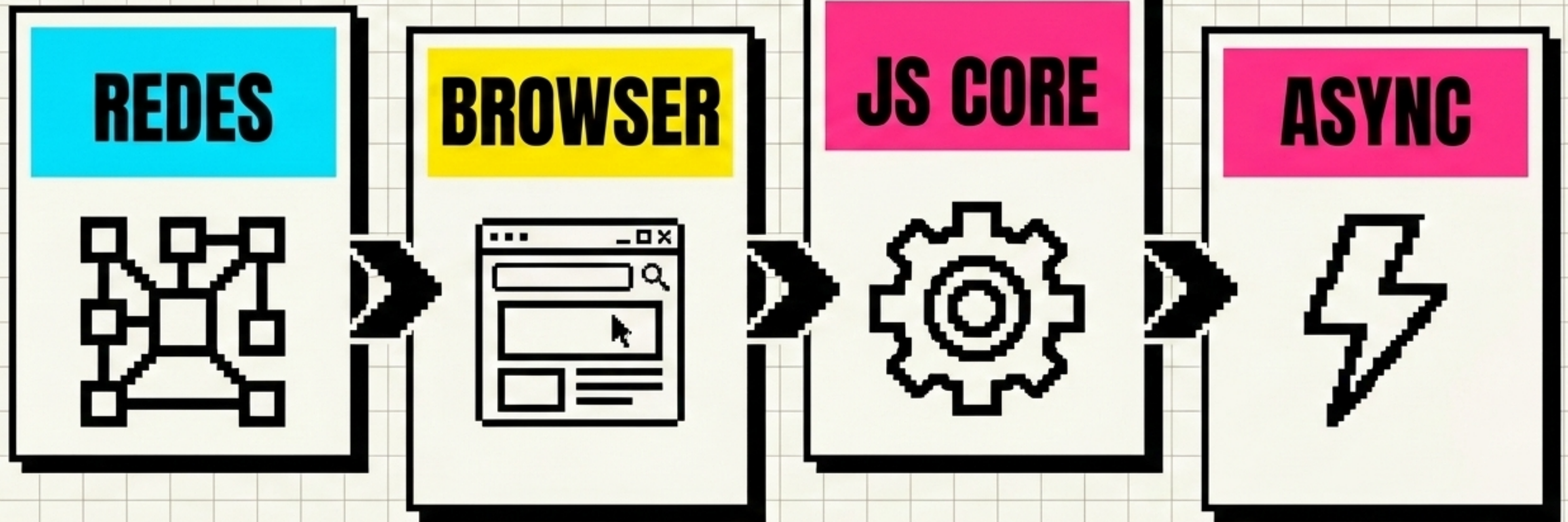


REPASO I1: TECNOLOGÍAS WEB

// ARCHITECTURE & SYSTEM REVIEW (IIC2513)



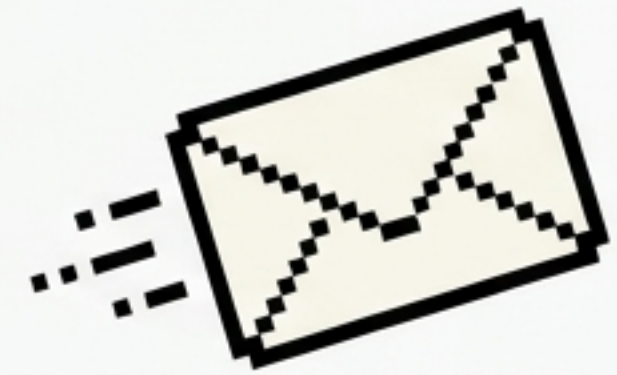
ENRUTAMIENTO VS. TRANSPORTE

// NETWORK LAYER VS TRANSPORT LAYER

TCP (Transmission Control Protocol): Confiabilidad absoluta. Handshake inicial, ACKs (acuses de recibo), reensamblaje de paquetes en orden estricto y puertos de aplicación.



IP (Internet Protocol): Enrutamiento best-effort. Lanza el paquete y olvida. No hay puertos, solo direcciones IP. No garantiza entrega.



LA BATALLA CONTRA LA LATENCIA

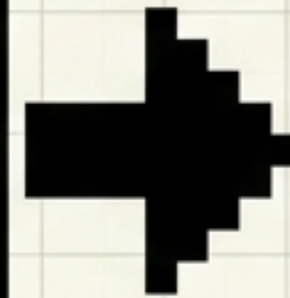
// HTTP PROTOCOL EVOLUTION

HTTP/1.1: Serial y basado en texto. Bloqueante por naturaleza.

Múltiples conexiones TCP necesarias.



HTTP/2: Binario y multiplexado. Una sola conexión TCP. Falla crítica: sufre de Head-of-Line (HoL) blocking a nivel de TCP.



HTTP/3: Construido sobre QUIC (UDP). Abandona TCP.

Resuelve el HoL blocking nativamente y reduce la latencia del handshake.

TAXONOMÍA DE RESPUESTAS

// HTTP STATUS CODES

1xx
(Informativo)

2xx
(Éxito)

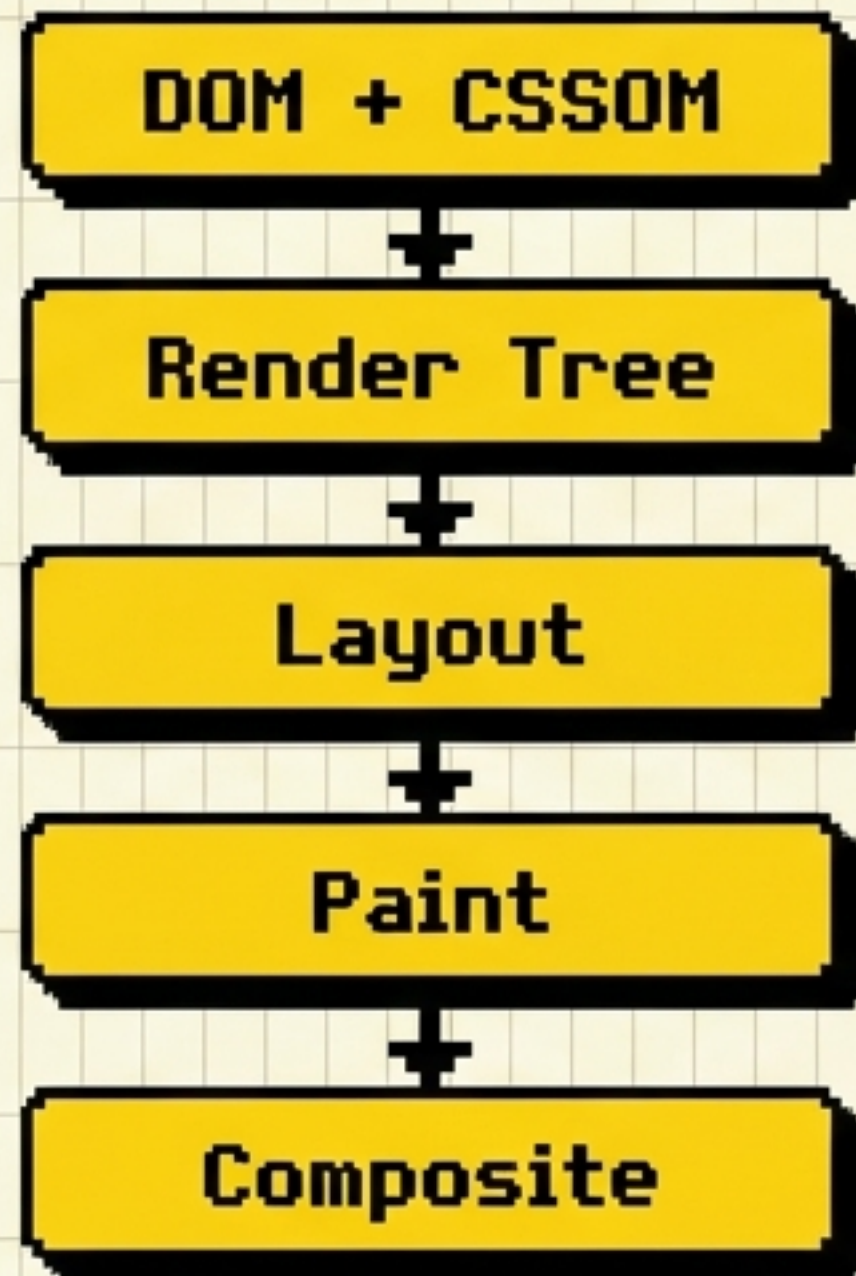
3xx
(Redirección)

4xx
**(Error del
Cliente)**

5xx
**(Error del
Servidor)**

DEL CÓDIGO AL PIXEL

// BROWSER RENDERING PIPELINE



display:none: el nodo sodo sigue en el DOM, pero NO entra al Render Tree (no ocupa espacio en el Layout).

visibility:hidden: SÍ entra al Render Tree, ocupa espacio en el Layout, simplemente no se pinta.



CUELLOS DE BOTELLA

// PARSER & RENDER BLOCKING

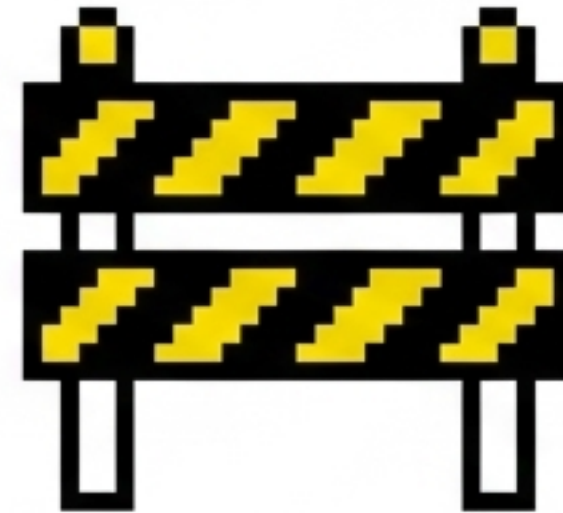
CSS: RENDER-BLOCKING



El navegador detiene el renderizado visual hasta construir el CSSOM.

Estrategia: Critical CSS inline.

JS: PARSER-BLOCKING

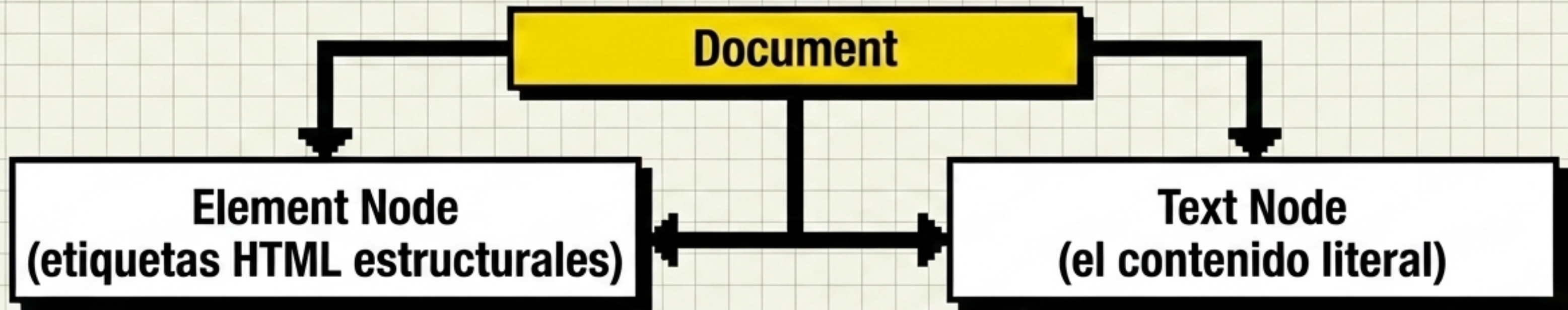


La ejecución síncrona detiene la construcción del DOM.

Estrategia: Atributos defer o async en el script.

EL ÁRBOL DE NODOS

// DOCUMENT OBJECT MODEL



MANIPULANDO EL ÁRBOL

`appendChild(nodo)`: agrega al final de los hijos (o mueve el nodo si ya existía)

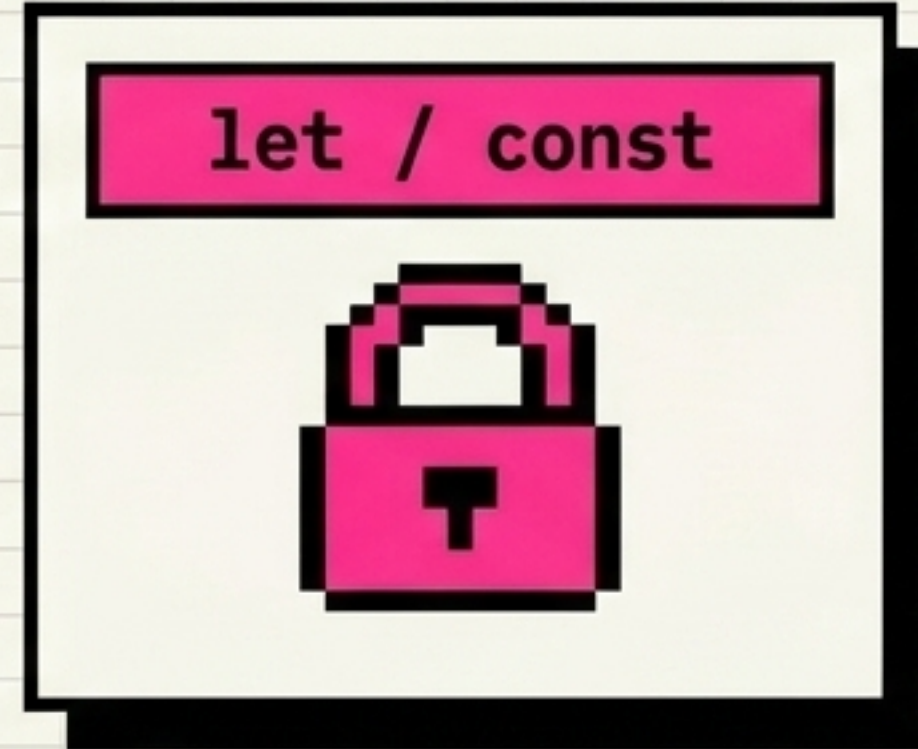
`insertBefore(nuevo, ref)`: inserta antes de un hijo específico

`removeChild(nodo)`: elimina un hijo del árbol

`textContent` = texto plano · `innerHTML` = interpreta HTML (riesgo XSS)

ELEVACIÓN Y ZONA MUERTA

// HOISTING & TEMPORAL DEAD ZONE



FUNCIONES

Declaración: `function hola(){} →`
se eleva COMPLETA. Se puede llamar
antes.

Expresión: `var chao = function(){} →`
solo se eleva la variable (undefined).
Llamarla antes = `TypeError`.

SCOPE EN BUCLES Y EVENTOS

// VAR SHARED BINDING VS LET PER-ITERATION

for(var i) + onclick

```
for (var i = 0; i < 6; i++) {  
  boton.onclick = function () {  
    console.log("Botón " + i);  
  };  
}
```

Cualquier botón muestra 6.



var tiene UNA sola variable por función. Los handlers corren después del bucle y todos leen el mismo valor final.

for(let i) + onclick

```
for (let i = 0; i < 6; i++) {  
  boton.onclick = function () {  
    console.log("Botón " + i);  
  };  
}
```

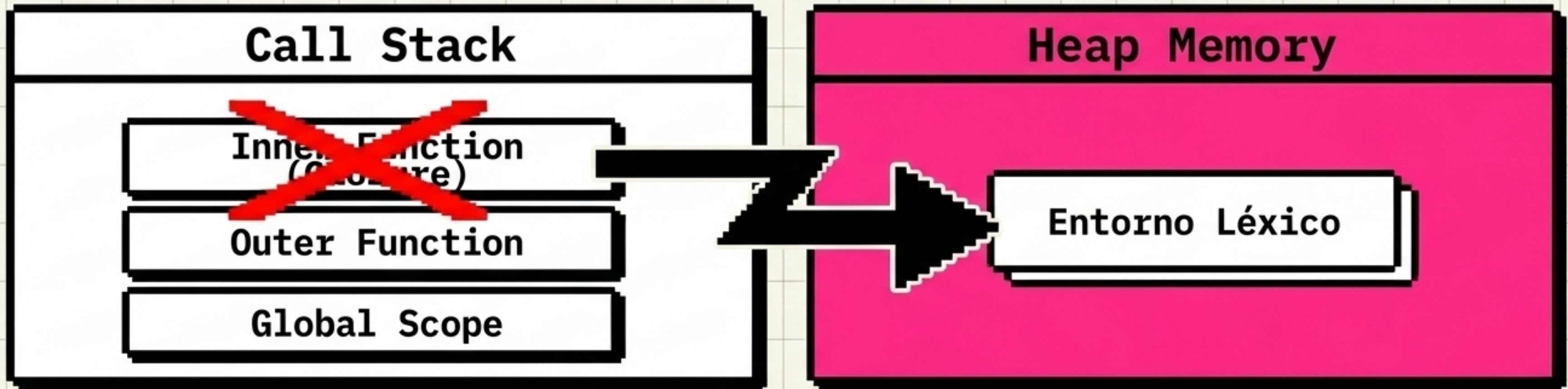
Botón 0 muestra 0,
Botón 1 muestra 1...
Botón 5 muestra 5.



let crea un binding nuevo en cada iteración. Cada handler captura su propio **i**.

MEMORIA PERSISTENTE

// LEXICAL ENVIRONMENT RETENTION



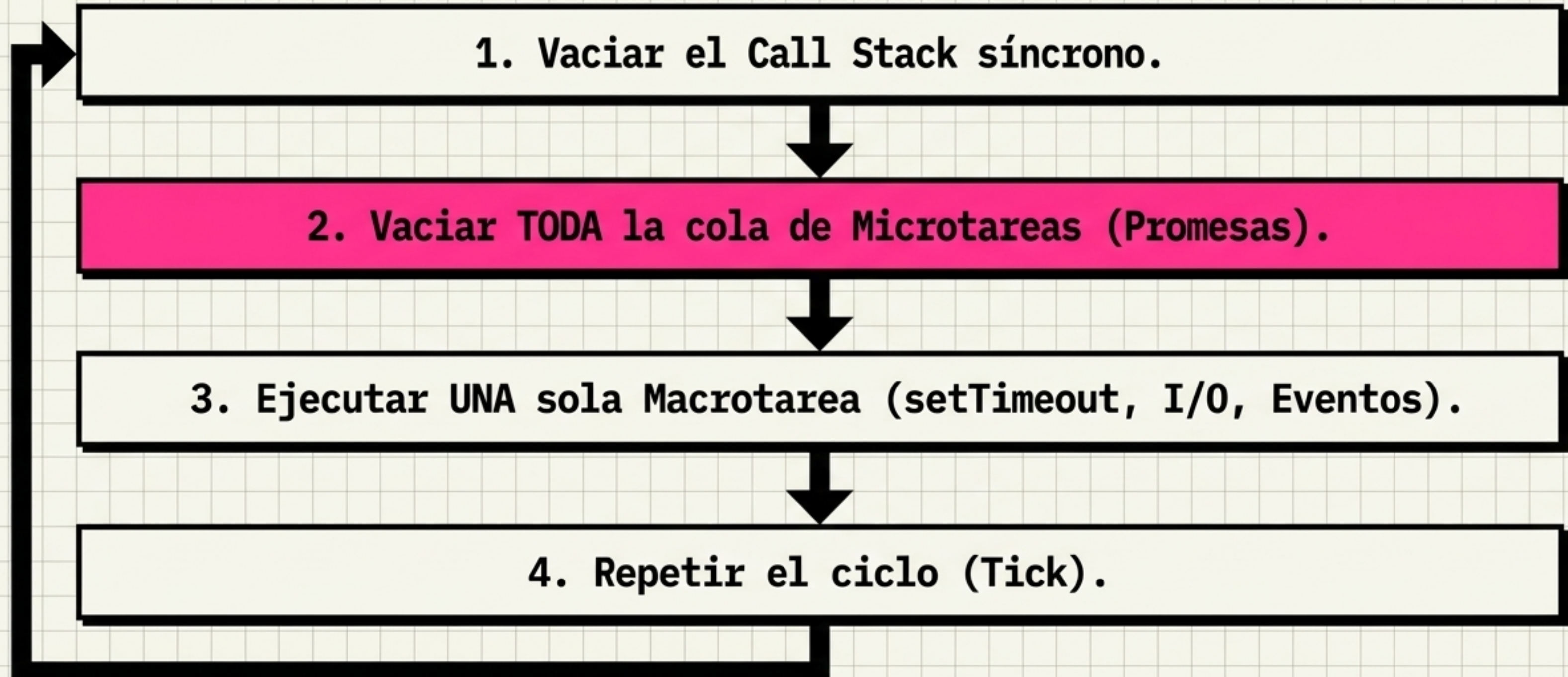
```
function crearMultiplicador(n) {  
  return function (x) { return x * n; };  
}  
const doble = crearMultiplicador(2);  
const triple = crearMultiplicador(3);  
doble(5); // 10  
triple(5); // 15
```

EJERCICIO

crearMultiplicador(2) ya terminó.
Explica qué hace posible que
doble(5) dé 10.

EL DICTADOR DEL HILO ÚNICO

// EVENT LOOP PRIORITY QUEUE



AUTOPSIA DE EJECUCIÓN

// ASYNC STEP-BY-STEP TRACE

Paso 1: código síncrono → corre de inmediato en el Call Stack.

Paso 2: Promise.then / queueMicrotask → cola de Microtarefas. Se vacía POR COMPLETO tras el código síncrono.

Paso 3: setTimeout → cola de Macrotareas. UNA por ciclo. Las microtarefas que genere se vacían antes de la siguiente macrotarea.

EJERCICIO: ¿QUÉ IMPRIME?

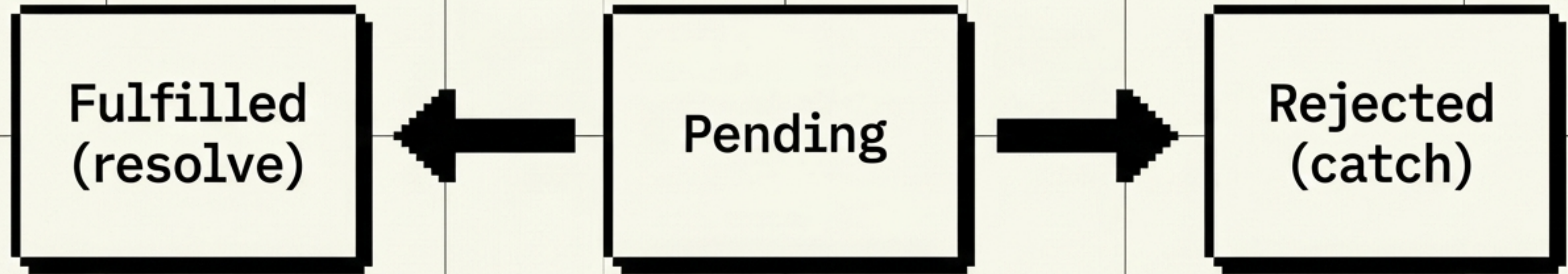
```
console.log("Abrir tienda");

setTimeout(() => console.log("Timeout A"), 0);
setTimeout(() => {
  console.log("Timeout B");
  Promise.resolve().then(() =>
    console.log("Micro dentro de B"));
}, 0);
Promise.resolve().then(() =>
  console.log("Micro X"));

queueMicrotask(() => console.log("Micro Y"));
console.log("Cerrar tienda");
```

EL CONTRATO FUTURO

// PROMISE LIFECYCLE & ENCADENAMIENTO



CALLBACKS

```
obtenerUsuario(id, function (u) {  
  obtenerPedidos(u.id, function (p) {  
    mostrar(p, function () { ... });  
  });  
});
```

PROMESAS

```
obtenerUsuario(id)  
  .then(u => obtenerPedidos(u.id))  
  .then(p => mostrar(p))  
  .catch(err => console.error(err));
```

ORQUESTACIÓN ASÍNCRONA

// PROMISE CONCURRENCY METHODS

Promise.all

Todo o nada. Se cumple cuando TODAS se cumplen; falla rápido si una se rechaza. Ejemplo: dashboard que necesita perfil, pedidos y notificaciones.

Promise.any

Supervivencia. Se cumple con la PRIMERA que tenga éxito. Ignora rechazos individuales; falla solo si todas fallan.

Promise.race

Gana la primera en terminar, sea éxito o error. Ejemplo: fetch vs timer para implementar un timeout.

DIAGNÓSTICO FINAL

// 11 CORE CONCEPTS CHECKLIST



HTTP/3 corre sobre QUIC/UDP: sigue siendo confiable, pero con streams independientes y handshake más rápido.



El Render Tree solo incluye nodos visibles: display:none queda fuera, visibility:hidden no.



Un closure combina una función con el entorno léxico donde fue creada.



JS tiene un solo hilo: el Event Loop coordina Call Stack, microtarefas y macrotareas.

END OF PROCESS

// Q&A / WAITING FOR INPUT...

